

Unit-5: Dynamic Programming

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to sub-problems. (“Programming” in this context refers to a tabular method, not to writing computer code.).
- Divide-and-conquer algorithms partition the problem into disjoint sub-problems, solve the sub-problems recursively, and then combine their solutions to solve the original problem. In contrast, **dynamic programming applies when the sub-problems overlap—that is, when sub-problems share sub-subproblems.**
- This paradigm is typically applied to **optimization problem.**
- **Dynamic programming is an algorithmic paradigm that solves complex problem by breaking it into subproblems and storing the results of sub problems to avoid computing the same results again.**
- **A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.**
- It is a powerful design technique for solving optimization problems. It is used when the sub-problems are not independent, e.g. when they share the same sub-problems. In this case, divide and conquer may do more work than necessary, because it solves the same sub-problem more than once. However, dynamic problem in this case solves each sub-problem just once and stores the result in a table so that it can be repeatedly retrieved if needed again.
- Dynamic programming is a Bottom-up approach. We solve all possible small results and then combine to obtain solutions for bigger problems.
- When developing a dynamic-programming algorithm, we follow a sequence of four steps:
 1. Characterize the structure of an optimal solution.
 2. Recursively define the value of an optimal solution.
 3. Compute the value of an optimal solution, typically in a bottom-up fashion.
 4. Construct an optimal solution from computed information.

Elements of dynamic programming

Dynamic programming is not applicable to all optimization problems. There are two important elements that a problem must have in order for DP to be applicable. The **two key ingredients** that an optimization problem must have in order for dynamic programming to apply are **optimal substructure** and **overlapping subproblems**.

- **Optimal substructure**

- The first step in solving an optimization problem by dynamic programming is to characterize the structure of an optimal solution. A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems. Whenever a problem exhibits optimal substructure, we have a good clue that dynamic programming might apply. In dynamic programming, we build an optimal solution to the problem from optimal solutions to subproblems.

- **Overlapping subproblems**

- The second ingredient that an optimization problem must have for dynamic programming to apply is that the space of subproblems must be “small” in the sense that a recursive algorithm for the problem solves the same subproblems over and over, rather than always generating new subproblems. Typically, the total number of distinct subproblems is a polynomial in the input size.
- When a recursive algorithm revisits the same problem repeatedly, we say that the optimization problem has **overlapping subproblems**. Dynamic-programming algorithms typically take advantage of overlapping subproblems by solving each subproblem once and then storing the solution in a table where it can be looked up when needed, using constant time per lookup.

Greedy paradigm vs Dynamic Programming paradigm

- Greedy and Dynamic Programming paradigm for algorithms are used to solve optimization problem.
- One major difference between greedy algorithms and dynamic programming is that instead of first finding optimal solutions to subproblems and then making an informed choice, greedy algorithms first make a greedy choice, the choice that looks best at the time, and then solve a resulting subproblem, without bothering to solve all possible related smaller subproblems.

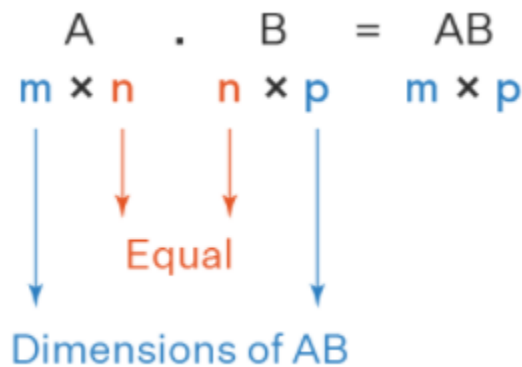
Greedy Paradigm	Dynamic Programming Paradigm
• It finds the feasible solution at every stage with the hope of finding an optimal solution.	• It breaks the problem into series of overlapping sub-problems with the hope of avoiding repetitive tasks.
• In a greedy Algorithm, we make whatever choice seems best at the moment and then solve the sub-problems arising after the choice is made.	• In Dynamic Programming, we choose at each step, but the choice may depend on the solution to sub-problems.
• In greedy method, local optimum solution is selected without considering the previously generated solution.	• Dynamic programming considers all possible sequences in order to obtain the optimum solution
• It takes the decision in one time.	• It takes the decision at every step.
• In Greedy Method, there is no guarantee of getting Optimal Solution.	• It is guaranteed that Dynamic Programming will generate an optimal solution using Principle of Optimality.
• Example: Fractional Knapsack problem, Kruskal's Algorithm for MST	• Example: Matrix Chain Multiplication, Zero-One Knapsack problem

Recursion vs Dynamic Programming

Recursion	Dynamic Programming
It is the repeated application of the same procedure on sub-problems of the same type of a problem.	It is the caching of the sub-problems so that every sub-problem is solved only once.
It solves identical sub-problems multiple times.	It uses previous results to avoid this repetition.
It stores values in stack.	It stores values in table.
The values are popped one by one, after certain condition.	The values are retrieved whenever required.
It follows top down approach	It follows bottom up approach

Matrix Chain Multiplication

— Concept of matrix multiplication :



- If matrix **A** has dimensions $m \times n$ and matrix **B** has dimensions $n \times q$, then matrix $C = A \times B$ will have dimensions $m \times q$, and will require $m \times n \times q$ scalar multiplications
- Matrix multiplication is associative. $[A \times (B \times C) = (A \times B) \times C]$
 - Matrix multiplication is associative, and so all parenthesizations yield the same product
- We can multiply only two matrices at a time.

Matrix Chain Multiplication

We are given a sequence (chain) $(A_1, A_2, A_3, \dots, A_n)$ of n matrices to be multiplied, and we wish to compute the product $A_1 A_2 A_3 \dots A_n$

- A product of matrices is fully parenthesized if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses.

- For example, if the chain of matrices is (A_1, A_2, A_3, A_4) , then we can fully parenthesize the product $A_1 A_2 A_3 A_4$ in five distinct ways:

$$\begin{aligned} &(A_1(A_2(A_3A_4))) \\ &(A_1((A_2A_3)A_4)) \\ &((A_1A_2)(A_3A_4)) \\ &((A_1(A_2A_3))A_4) \\ &(((A_1A_2)A_3)A_4) \end{aligned}$$

Example:

Consider a problem of chain (A_1, A_2, A_3) of three matrices. Suppose that the dimension of A_1 is 10×100 , A_2 is 100×5 and A_3 is 5×50

Possible parenthesizations and number of scalar multiplications needed

— $((A_1 A_2)A_3)$

$$10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500 \text{ scalar multiplications}$$

— $((A_1 (A_2 A_3))$

$$10 \times 5 \times 50 + 10 \times 100 \times 50 = 75000 \text{ scalar multiplications}$$

Computing the product according to the first parenthesization is 10 times faster.

Matrix-chain multiplication problem

Given a chain $(A_1, A_2, A_3, \dots, A_n)$, where for $i = 1, 2, 3, \dots, n$ matrix A_i has dimension $p_{i-1} \times p_i$ fully parenthesize the product $A_1 A_2 A_3 \dots A_n$ (i.e. determine the order of multiplication) in a way that minimizes the number of scalar multiplications.

Note: In the matrix-chain multiplication problem, we are **not actually multiplying matrices**. Our goal is only to determine an order for multiplying matrices that has the lowest cost.

Recursive definition of optimal solution: Let $m[i, j]$ denotes minimum number of scalar multiplications needed to compute $A_i \dots A_j$.

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

The $m[j,j]$ values give the costs of optimal solutions to subproblems, but they do not provide all the information we need to construct an optimal solution. To help us do so, we define $s[j,j]$ to be a value of k at which we split the product $A_i A_{i+1} A_j$ in an optimal parenthesization. That is, $s[j,j]$ equals a value k such that

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

Algorithm

Matrix-Chain-Order(p)

```

{
    n = p.length - 1;
    for( i = 1; i <= n; i++)
    {
        m[i, i] = 0;
    }
    for(l = 2; l <= n; l++) // l is the chain length
    {
        for(i = 1; i <= n - l + 1; i++)
        {
            j = i + l - 1;
            m[i, j] = ∞;
            for(k = i; k <= j - 1; k++)
            {
                c = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j];
                if c < m[i, j]
                {
                    m[i, j] = c;
                    s[i, j] = k;
                }
            }
        }
    }
    return m and s
}

```

Analysis

The above algorithm can be easily analyzed for running time as $O(n^3)$, due to three nested loops. The space complexity is $O(n^2)$ to store the m and s tables.

Note:

- Although Matrix-Chain-Order determines the optimal number of scalar multiplications needed to compute a matrix-chain product, it does not directly show how to multiply the matrices. The table s gives us the information we need to do so. Each entry $s[i,j]$ records a value of k such that an optimal parenthesization of $A_i A_{i+1} A_j$ splits the product between A_k and A_{k+1} .
- The following recursive procedure prints an optimal parenthesization of $A_i, A_{i+1}, \dots, A_j$ given the s table computed by Matrix-Chin-Order and the indices i and j .
- The initial call PRINT-OPTIMAL-PARENS(s,1,n) prints an optimal parenthesization of
- $(A_i, A_{i+1}, \dots, A_n)$.

PRINT-OPTIMAL-PARENS (s, i, j)

```

1  if i == j
2      print "A"i
3  else print "("
4      PRINT-OPTIMAL-PARENS (s, i, s[i, j])
5      PRINT-OPTIMAL-PARENS (s, s[i, j] + 1, j)
6      print ")"
```

Example:

Given a chain of matrix $A_1 (5 \times 4)$. $A_2 (4 \times 6)$. $A_3 (6 \times 2)$. $A_4 (2 \times 7)$. Compute optimum order of multiplication (paranthesization) for this chain using dynamic programming.

Solution:-

Given $p[] = \{5,4,6,2,7\}$ // represents dimensions of matrices

We have to maintain two tables m and s to store the results. m is used to store number of scalar multiplication (cost) and s is used to store the position of parenthesis.

m

i \ j	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

s

i \ j	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

When $l = 1$,

$m[i,j] = 0$ for $i = j$

So all diagonal elements of m are set to 0.

For $l = 2$ to 4 ($l = 2$ to $n = p.length-1$)

When $l = 2$,

For $i = 1$ to $(n-l+1) = 4-2+1 = 3$

When $i = 1$

$$j = i+l-1 = 1+2-1 = 2$$

$i = 1, j = 2$, compute $m[1,2]$ in $k = 1$ to 1 (i.e. i to $j-1$)

$$m[1,2] = m[1,1] + m[2,2] + p[0].p[1].p[2]$$

$$= 0 + 0 + 5 \times 4 \times 6 = 120$$

Minimum value of $m[1,2]$ among all possible k values is selected to fill in $m[1,2]$ and respective k value is selected to fill in $s[1,2]$. Hence $m[1,2] = 120$ and $s[1,2] = 1$

When $i = 2$

$$j = i+l-1 = 2+2-1 = 3$$

$i = 2, j = 3$, compute $m[2,3]$ in $k = 2$ to 2 (i.e. i to $j-1$)

$$m[2,3] = m[2,2] + m[3,3] + p[1].p[2].p[3]$$

$$= 0 + 0 + 4 \times 6 \times 2 = 48$$

Minimum value of $m[2,3]$ among all possible k values is selected to fill in $m[2,3]$ and respective k value is selected to fill in $s[2,3]$. Hence $m[2,3] = 48$ and $s[2,3] = 2$

When $i = 3$

$$j = i+l-1 = 3+2-1 = 4$$

$i = 3, j = 4$, compute $m[3,4]$ in $k = 3$ to 3 (i.e. i to $j-1$)

$$\begin{aligned} m[3,4] &= m[3,3] + m[4,4] + p[2].p[3].p[4] \\ &= 0 + 0 + 6 \times 2 \times 7 = 84 \end{aligned}$$

Minimum value of $m[2,3]$ among all possible k values is selected to fill in $m[3,4]$ and respective k value is selected to fill in $s[3,4]$. Hence $m[3,4] = 48$ and $s[3,4] = 3$

When $l = 3$,

$$\text{For } i = 1 \text{ to } (n-l+1) = 4-3+1 = 2$$

When $i = 1$

$$j = i+l-1 = 1+3-1 = 3$$

$i = 1, j = 3$, compute $m[1,3]$ in $k = 1$ to 2 (i.e. i to $j-1$)

$k = 1$	$m[1,3] = m[1,1] + m[2,3] + p[0].p[1].p[3]$ $= 0 + 48 + 5 \times 4 \times 2 = 88$	Minimum value of $m[1,3]$ is 88 when $k = 1$ So. $m[1,3] = 88$ $s[1,3] = 1$
$k = 2$	$m[1,3] = m[1,2] + m[3,3] + p[0].p[2].p[3]$ $= 0 + 120 + 5 \times 6 \times 2 = 180$	

When $i = 2$

$$j = i+l-1 = 2+3-1 = 4$$

$i = 2, j = 4$, compute $m[2,4]$ in $k = 2$ to 3 (i.e. i to $j-1$)

$k = 2$	$m[2,4] = m[2,2] + m[3,4] + p[1].p[2].p[4]$ $= 0 + 84 + 4 \times 6 \times 7 = 252$	Minimum value of $m[2,4]$ is 104 when $k = 3$ So. $m[2,4] = 104$ $s[2,4] = 3$
$k = 3$	$m[2,4] = m[2,3] + m[4,4] + p[1].p[3].p[4]$ $= 48 + 0 + 4 \times 2 \times 7 = 104$	

When $l = 4$,

$$\text{For } i = 1 \text{ to } (n-l+1) = 4-4+1 = 1$$

When $i = 1$

$$j = i+l-1 = 1+4-1 = 4$$

$i = 1, j = 4$, compute $m[1,4]$ in $k = 1$ to 3 (i.e. i to $j-1$)

$k = 1$	$m[1,4] = m[1,1] + m[2,4] + p[0].p[1].p[4]$ $= 0 + 104 + 5 \times 4 \times 7 = 244$	Minimum value of $m[1,4]$ is 158 when $k = 3$ So, $m[1,4] = 158$ $s[1,3] = 3$
$k = 2$	$m[1,4] = m[1,2] + m[3,4] + p[0].p[2].p[4]$ $= 120 + 84 + 5 \times 6 \times 7 = 414$	
$k = 3$	$m[1,4] = m[1,3] + m[4,4] + p[0].p[3].p[4]$ $= 88 + 0 + 5 \times 2 \times 7 = 158$	

From m table, $m[1,4] = 158$ gives the optimum total number of multiplication required to multiply the given chain of matrices.

Now from table s , we can parenthesize the chain as

$A_1 A_2 A_3 A_4$

$(A_1 A_2 A_3) (A_4)$ [since $s[1,4] = 3$ we should parenthesize from A_1 to A_3 , A_4 is separated]

$(A_1 A_2 A_3) (A_4)$ [since $s[1,3] = 1$ we should parenthesize from A_1 to A_1 , A_2 and A_3 is separated]

$((A_1)(A_2 A_3)) (A_4)$

Which is the optimum parenthesization of given chain of matrices.

Exercise:

Given a chain of matrix $A_1 (2 \times 5)$, $A_2 (5 \times 3)$, $A_3 (3 \times 5)$, $A_4 (5 \times 2)$, $A_5 (2 \times 3)$, $A_6 (3 \times 5)$. Compute optimum order of multiplication (parenthesization) for this chain using dynamic programming.

0-1 Knapsack Problem

- Given a knapsack with maximum capacity w and set of n items with different weights w_i and values v_i for each item i . How to pack the knapsack to achieve maximum value using 0-1 strategy i.e. either take whole item or leave whole item.
- Given weights and values of n items, put these items in a knapsack of capacity w to get the maximum total value in the knapsack and the item taken should be whole.
- Straight forward method to solve this problem:
 - group n items into 2^n possible groups then select the group with maximum total values.
 - Complexity : $O(2^n)$

Solving 0-1 Knapsack problem using Dynamic Programming Approach

- Inputs to the algorithm are maximum weight W , the number of items n , two arrays $v[]$ for values of items and $w[]$ for weight of items.
- Let us assume that the table $c[i, w]$ is the value of solution for items 1 to i and maximum weight w . Then we can define recurrence relation for 0/1 knapsack problem as:

$$c[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ c[i - 1, w] & \text{if } w_i > w \\ \max\{c[i - 1, w], c[i - 1, w - w_i] + v[i]\} & \text{if } i > 0 \text{ and } w \geq w_i \end{cases}$$

- The above relation expresses that:
 - If we have no items or the maximum weight is 0 then the solution is 0.
 - If weight of item exceeds the total weight remaining then the solution is the solution of previous item addition.
 - If total item remaining is greater than or equal to the weight of item being added, the solution is the maximum value between the sum of value up to previous addition of items or after addition of item

The process can be represented in following pseudocode

```

DynaKnapsack(W,n,v,w)
{
    for(w=0; w<=W; w++)
        c[0,w] = 0;
    for(i=1; i<=n; i++)
        c[i,0] = 0;
    for(i=1; i<=n; i++)
    {
        for(w=1; w<=W; w++)
        {
            if(w[i]< w)
            {
                if ((v[i] +c[i-1,w-w[i]] > c[i-1,w])
                {
                    c[i,w] = v[i] +c[i-1,w-w[i]];
                }
                else
                {
                    c[i,w] = c[i-1,w];
                }
            }
            else
            {
                c[i,w] = c[i-1,w];
            }
        }
    }
}

```

Analysis

- The first loop that checks the weight of item and weight of knapsack runs in $O(W)$
- The second loop to check the number of items takes $O(n)$
- The third loop is executed at most n times and inside this, another loop executes for at most W times where the items chosen to put into knapsack is achieved. So, this inner loop runs in $O(W)$. The time complexity for this loop (outer) is $O(nW)$
- Therefore the total time complexity for this algorithm is $O(nW)$

Algorithm to find items to be added into the knapsack to achieve the optimum value generated from above dynamic approach can be represented as:

- Start from $c[n, W]$
 - Let $i = n, w = W$
- While($i > 0$ and $w > 0$)
 - If $(c[i, w] \neq c[i-1, w])$ then
 - Mark i^{th} item to be added to knapsack
 - $w = w - w[i], i = i-1$
 - else
 - $i = i-1$

Note:

To trace the algorithm, a table representing the values should be created.

If W is the capacity of knapsack and n is the number of items then

- Number of columns of the table = $W + 1$ [from 0 to W]
- Number of rows of the table = $n + 1$ [from 0 to n]

Example:

Items	1	2	3	4
Weights	2	3	4	5
Values	3	4	5	6

Knapsack capacity (W): 5

Solution:

The weight array $w[] = \{0, 2, 3, 4, 5\}$

The value array $v[] = \{0, 3, 4, 5, 6\}$

To trace the algorithm we have create a table c having columns (0 to knapsack capacity (W)) and rows (0 to number of items (n)).

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

For $i = 0$ and $w = 0$, $c[i, w] = 0$

For $i = 1$, compute $c[1, w]$ for $w = 1$ to 5

$W[i] = w[1] = 2$ and $v[i] = v[1] = 3$

When $w = 1$

$$w[i] > w$$

$$\text{So, } c[1, 1] = c[0, 1] = 0$$

When $w = 2$

$$w[i] = w$$

$$\begin{aligned} \text{So, } c[1, 2] &= \max\{c[0, 2], c[0, 2-2] + v[1]\} \\ &= \max\{0, 0 + 3\} = 3 \end{aligned}$$

When $w = 3$

$$w > w[i]$$

$$\begin{aligned} \text{So, } c[1, 3] &= \max\{c[0, 3], c[0, 3-2] + v[1]\} \\ &= \max\{c[0, 3], c[0, 1] + v[1]\} \\ &= \max\{0, 0 + 3\} = 3 \end{aligned}$$

When $w = 4$

$$w > w[i]$$

$$\begin{aligned} \text{So, } c[1, 4] &= \max\{c[0, 3], c[0, 4-2] + v[1]\} \\ &= \max\{c[0, 3], c[0, 2] + v[1]\} \\ &= \max\{0, 0 + 3\} = 3 \end{aligned}$$

When $w = 5$

$$w > w[i]$$

$$\begin{aligned}\text{So, } c[1,4] &= \max\{ c[0,3], c[0,5-2] + v[1] \} \\ &= \max\{ c[0,3], c[0,3] + v[1] \} \\ &= \max\{ 0, 0 + 3 \} = 3\end{aligned}$$

For $i = 2$, compute $c[2,w]$ for $w = 1$ to 5

$$W[i] = w[2] = 3 \text{ and } v[i] = v[2] = 4$$

When $w = 1$

$$w[i] > w$$

$$\text{So, } c[2,1] = c[1,1] = 0$$

When $w = 2$

$$w[i] > w$$

$$\text{So, } c[2,2] = c[1,2] = 3$$

When $w = 3$

$$w = w[i]$$

$$\begin{aligned}\text{So, } c[2,3] &= \max\{ c[1,3], c[1,3-3] + v[2] \} \\ &= \max\{ c[1,3], c[1,0] + v[2] \} \\ &= \max\{ 3, 0 + 4 \} = 4\end{aligned}$$

When $w = 4$

$$w > w[i]$$

$$\begin{aligned}\text{So, } c[2,4] &= \max\{ c[1,4], c[1,4-3] + v[2] \} \\ &= \max\{ c[1,4], c[1,1] + v[2] \} \\ &= \max\{ 3, 0 + 4 \} = 4\end{aligned}$$

When $w = 5$

$$w > w[i]$$

$$\begin{aligned}\text{So, } c[2,5] &= \max\{ c[1,5], c[1,5-3] + v[2] \} \\ &= \max\{ c[1,4], c[1,2] + v[2] \} \\ &= \max\{ 3, 3 + 4 \} = 7\end{aligned}$$

For $i = 3$, compute $c[3, w]$ for $w = 1$ to 5

$W[i] = w[3] = 4$ and $v[i] = v[3] = 5$

When $w = 1$

$$w[i] > w$$

$$\text{So, } c[3, 1] = c[2, 1] = 0$$

When $w = 2$

$$w[i] > w$$

$$\text{So, } c[3, 2] = c[2, 2] = 3$$

When $w = 3$

$$w[i] > w$$

$$\text{So, } c[3, 3] = c[2, 3] = 4$$

When $w = 4$

$$w = w[i]$$

$$\begin{aligned} \text{So, } c[3, 4] &= \max\{ c[2, 4], c[2, 4-4] + v[3] \} \\ &= \max\{ c[2, 4], c[2, 0] + v[3] \} \\ &= \max\{ 4, 0 + 5 \} = 5 \end{aligned}$$

When $w = 5$

$$w > w[i]$$

$$\begin{aligned} \text{So, } c[3, 5] &= \max\{ c[2, 5], c[2, 5-4] + v[3] \} \\ &= \max\{ c[2, 5], c[2, 1] + v[3] \} \\ &= \max\{ 7, 0 + 5 \} = 7 \end{aligned}$$

For $i = 4$, compute $c[4, w]$ for $w = 1$ to 5

$W[i] = w[4] = 5$ and $v[i] = v[4] = 6$

When $w = 1$

$$w[i] > w$$

$$\text{So, } c[4, 1] = c[3, 1] = 0$$

When $w = 2$

$$w[i] > w$$

$$\text{So, } c[4,2] = c[3,2] = 3$$

When $w = 3$

$$w[i] > w$$

$$\text{So, } c[4,3] = c[3,3] = 4$$

When $w = 4$

$$w[i] > w$$

$$\text{So, } c[4,4] = c[3,4] = 5$$

When $w = 5$

$$w = w[i]$$

$$\begin{aligned} \text{So, } c[4,5] &= \max\{ c[3,5], c[3,5-5] + v[4] \} \\ &= \max\{ c[3,5], c[3,1] + v[4] \} \\ &= \max\{ 7, 0 + 6 \} = 7 \end{aligned}$$

Hence the maximum total value in knapsack = $c[4,5] = 7$

To find items to be added

Start from $c[4,5]$

Here $i = 4, w = 5$

$$c[4,5] = 7 \text{ and } c[3,5] = 7$$

$$\text{i.e. } c[4,5] = c[3,5]$$

$$\text{So, } i = i-1 = 4-1 = 3, w = 5$$

$$c[3,5] = 7, c[2,5] = 7$$

$$\text{i.e. } c[3,5] = c[2,5]$$

$$\text{So, } i = i-1 = 3-1 = 2, w = 5$$

$$c[2,5] = 7, c[1,5] = 5$$

$$\text{i.e. } c[2,5] \neq c[1,5]$$

so, mark item 2

$$w = w - w[i] = 5 - w[2] = 5 - 3 = 2$$

$$i = i-1 = 2-1 = 1$$

Now,

$$c[1,2] = 3 \text{ and } c[0,2] = 0$$

$$\text{i.e. } c[1,2] \neq c[0,2]$$

so, mark item 1

$$w = w - w[i] = w - w[1] = 2-2 = 0$$

$$i = i-1 = 1-1 = 0$$

since $i = 0$ and $w = 0$ stop.

Hence item to be added into knapsack = {2, 1}

$$\text{Total profit} = 5+2 = 7$$

The Floyd-Warshall algorithm for all pairs shortest path problem

- The algorithm is used to compute the all pairs shortest path in a weighted connected graph
- The algorithm follows dynamic programming approach
- Consider a weighted graph $G = (V, E)$ and denote the weight of edge connecting vertices i and j as w_{ij} . So, W represents the adjacency matrix for the given graph G .
- We want to compute the distance matrix $D[n, n]$ where n is the number of nodes such that $D[i, j]$ represents the shortest paths from node i to node j .
- Let D^k denote an $n \times n$ matrix such that $D^k[i, j]$ represents the shortest distance between vertex i to vertex j and represented as :

$D^k[i, j]$ = The weight of shortest path from i to j that may include 1, 2, 3, ..., k nodes as intermediate vertices.

Case 1: $D^k[i, j]$ does not include k as intermediate vertex then

$$D^k[i, j] = D^{k-1}[i, j]$$

Case 2: $D^k[i, j]$ includes k as intermediate vertex then

$$D^k[i, j] = \min\{ D^{k-1}[i, j], D^{k-1}[i, k] + D^{k-1}[k, j] \}$$

- So, the recursive formula is :

$$D^k[i, j] = \min\{ D^{k-1}[i, k] + D^{k-1}[k, j], D^{k-1}[i, j] \}$$

This relation is used by Floyd's algorithm to compute all pairs shortest path problem in bottom up manner finding $D^1, D^2, D^3, \dots, D^n$.

NOTE: This algorithm can handle negative weights but cannot handle negative cycles.

Algorithm

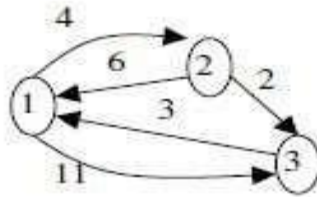
//W is weight adjacency matrix, D is Shortest distance matrix, n is number of vertices

```
FloydWarshallAPSP(W,D,n) {
    for( i = 1; i<=n; i++){
        for(j=1; j<=n; j++){
            D[i][j] = W[i][j]; // initialization as D0
        }
    }
    for(k=1; k<=n;k++){
        for(i=1; i<=n;i++){
            for(j=1; j<=n;j++){
                if(D[i][j]> D[i][k]+D[k][j]){
                    D[i][j] = D[i][k]+D[k][j]}
            }
        }
    }
}
```

Analysis

- Initializing distance matrix D^0 takes $O(n^2)$
- Computing final distance matrix takes $O(n^3)$ since the algorithm has three nested loops.
- So, the time complexity of the algorithm is $O(n^3)$

Example:



Solution:

Here, $D^0 = W$

Adjacency Matrix

W	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

D^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

D^2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

Remember we are not showing $D^k(i,i)$, since there will be no change i.e. shortest path is zero.

$$D^1(1,2) = \min\{D^0(1,2), D^0(1,1) + D^0(1,2)\}$$

$$= \min\{4, 0 + 4\} = 4$$

$$D^1(1,3) = \min\{D^0(1,3), D^0(1,1) + D^0(1,3)\}$$

$$= \min\{11, 0 + 11\} = 11$$

$$D^1(2,1) = \min\{D^0(2,1), D^0(2,1) + D^0(1,1)\}$$

$$= \min\{6, 6 + 0\} = 6$$

$$D^1(2,3) = \min\{D^0(2,3), D^0(2,1) + D^0(1,3)\}$$

$$= \min\{2, 6 + 11\} = 2$$

$$D^1(3,1) = \min\{D^0(3,1), D^0(3,1) + D^0(1,1)\}$$

$$= \min\{3, 3 + 0\} = 3$$

$$D^1(3,2) = \min\{D^0(3,2), D^0(3,1) + D^0(1,2)\}$$

$$= \min\{\infty, 3 + 4\} = 7$$

$$D^2(1,2) = \min\{D^1(1,2), D^1(1,2) + D^1(2,2)\}$$

$$= \min\{4, 4 + 0\} = 4$$

$$D^2(1,3) = \min\{D^1(1,3), D^1(1,2) + D^1(2,3)\}$$

$$= \min\{11, 4 + 2\} = 6$$

$$D^2(2,1) = \min\{D^1(2,1), D^1(2,2) + D^1(2,1)\}$$

$$= \min\{6, 0 + 6\} = 6$$

$$D^2(2,3) = \min\{D^1(2,3), D^1(2,2) + D^1(2,3)\}$$

$$= \min\{2, 0 + 2\} = 2$$

$$D^2(3,1) = \min\{D^1(3,1), D^1(3,2) + D^1(2,1)\}$$

$$= \min\{3, 7 + 6\} = 3$$

$$D^2(3,2) = \min\{D^1(3,2), D^1(3,2) + D^1(2,2)\}$$

$$= \min\{7, 7 + 0\} = 7$$

D^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

$$\begin{aligned}
 D^3(1,2) &= \min\{D^2(1,2), D^2(1,3) + D^2(3,2)\} \\
 &= \min\{4, 6 + 7\} = 4 \\
 D^3(1,3) &= \min\{D^2(1,3), D^2(1,2) + D^2(2,3)\} \\
 &= \min\{6, 4 + 0\} = 6 \\
 D^3(2,1) &= \min\{D^2(2,1), D^2(2,3) + D^2(3,1)\} \\
 &= \min\{5, 2 + 3\} = 5 \\
 D^3(2,3) &= \min\{D^2(2,3), D^2(2,1) + D^2(1,3)\} \\
 &= \min\{2, 5 + 6\} = 2 \\
 D^3(3,1) &= \min\{D^2(3,1), D^2(3,2) + D^2(2,1)\} \\
 &= \min\{3, 7 + 5\} = 3 \\
 D^3(3,2) &= \min\{D^2(3,2), D^2(3,3) + D^2(3,1)\} \\
 &= \min\{7, 0 + 3\} = 7
 \end{aligned}$$

The distance matrix D^4 gives the shortest paths cost between each pair of vertices.

Pairs	Shortest Path Cost
(1 to 1)	0
(1 to 2)	4
(1 to 3)	6
(2 to 1)	5
(2 to 2)	0
(2 to 3)	2
(3 to 1)	3
(3 to 2)	7
(3 to 3)	0

NOTE: While Creating Matrix D^k from D^{k-1}

- k^{th} row and k^{th} column remains same as in D^{k-1}
- if $(D^{k-1}[i][j] > (D^{k-1}[i][k] + A^{k-1}[k][j]))$

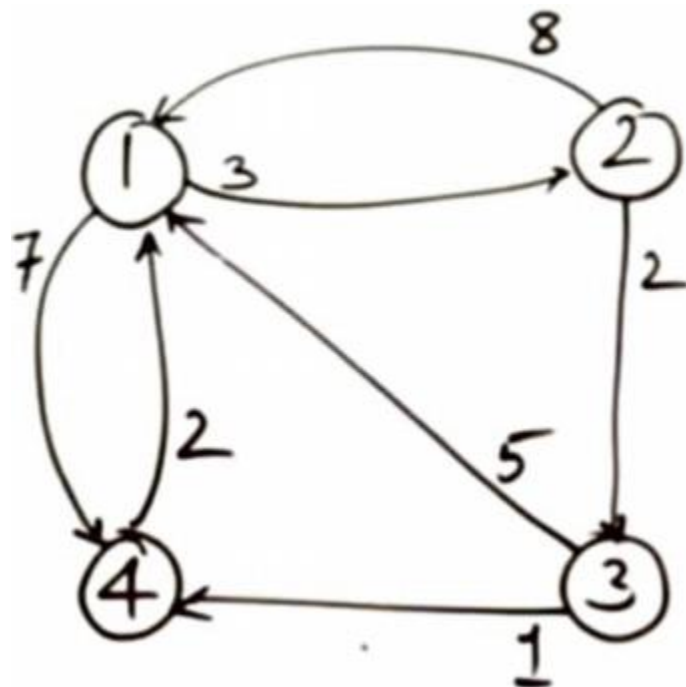
$$A^k[i][j] = D^{k-1}[i][k] + D^{k-1}[k][j]$$

else

$$D^k[i][j] = D^{k-1}[i][j]$$

$$\text{i.e. } D^k[i][j] = \min \{ D^{k-1}[i][j], (D^{k-1}[i][k] + D^{k-1}[k][j]) \}$$

Exercise: Find the cost of all pair shortest paths of given graph using Floyd Warshall algorithm.



Final Answer:

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

Travelling salesman problem

- The travelling salesman problem is also called a the travelling salesperson problem or TSP)
- Problem: "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"
- In other words, the problem is: "Given a complete graph with weighted edges (as adjacency matrix) what is the Hamiltonian cycle (path that visits every node once) of minimum cost?"
- There is non-negative cost $C(i,j)$ to travel from city i to j .
- The problem can be solved by using dynamic programming approach or by using other approach but here we discuss about dynamic programming approach.
- Let v be the starting vertex, S be the set of remaining vertices except v , k be any one vertex of set S , C_{ik} be the cost of travelling from vertex i to k and $g(i, S)$ be the minimum travel cost of TSP then the recurrence relation can be defined as follow:

$$g(i, S) = \min_{k \in S} \{C_{ik} + g(k, S - \{k\})\}$$

$$g(i, \Phi) = C_{iv}$$

Algorithm

//s= starting vertex, N= set of vertices

```

DPTSP(s, N) {
    visited[N] = 0;
    cost = 0;
    visited[s] = 1;
    if(|N| = 2 and k ≠ s) {
        cost(s, N) = dis[k, i]; // k ∈ S
        cost = cost(i, N);
        return cost;
    }
    else {
        for each j in N {
            for each i in N and visited[i] = 0 {
                if(j ≠ i and j ≠ s) {
                    cost(j, N) = min(DPTSP(j, N - {i} + dist[j, i])
                    visited[j] = 1;
                    cost = cost(j, N);
                }
            }
        }
    }
    return cost;
}

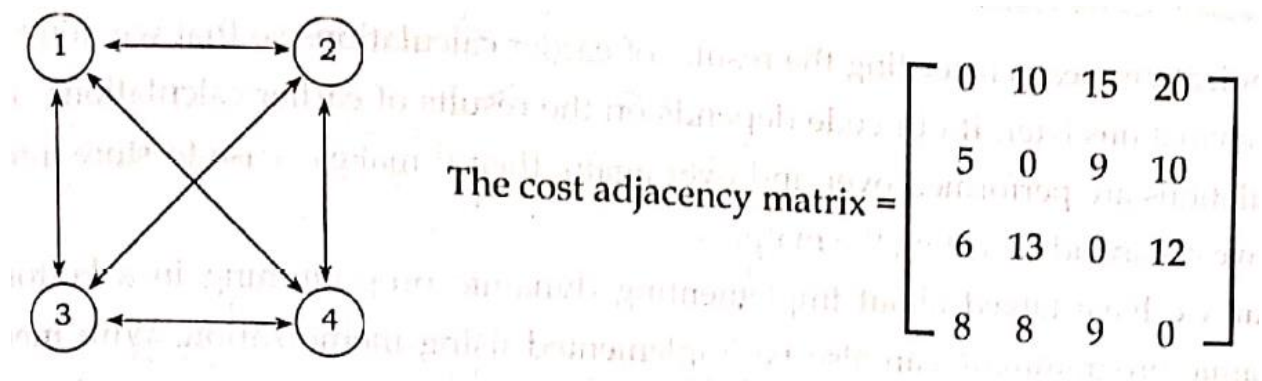
```


Analysis

- While computing $g(i, S)$ there are n possible 'i' s (i.e. vertices) and 2^n possible 'S' (sub sets of nodes)
- Total number of sub problems = $2^n n$
- Time complexity per sub problems = $O(n)$
- Total time complexity = $O(n^2 2^n)$

Note: The algorithm is known as Held–Karp algorithm

Example



In general we have to compute $g(i, S) = \min_{k \in S} \{C_{ik} + g(k, S - \{k\})\}$

Let's start the tour from start vertex 1:

Start vertex $s = 1$, Set of vertices $V = \{1, 2, 3, 4\}$

Set of vertices except start vertex x is $S = V - \{s\} = V - \{1\} = \{2, 3, 4\}$

So, we have to compute

$$g(1, S) = g(1, \{2, 3, 4\}) = \min_{k \in S} \{C_{1k} + g(k, \{2, 3, 4\} - \{k\})\}$$

Let's first compute the cost to reach the start vertex 1 from all other vertices.

(i.e when $S = \Phi$)

$$g(2, \Phi) = C_{21} = 5$$

$$g(3, \Phi) = C_{31} = 6$$

$$g(4, \Phi) = C_{41} = 8$$

Now let's compute g when S has 1 element

$$g(2, \{3\}) = C_{23} + g(3, \Phi) = 9 + 6 = 15$$

$$g(2, \{4\}) = C_{24} + g(4, \Phi) = 10 + 8 = 18$$

$$g(3, \{2\}) = C_{32} + g(2, \Phi) = 13 + 5 = 18$$

$$g(3, \{4\}) = C_{34} + g(4, \Phi) = 12 + 8 = 20$$

$$g(4, \{2\}) = C_{42} + g(2, \Phi) = 8 + 5 = 13$$

$$g(4, \{3\}) = C_{43} + g(3, \Phi) = 9 + 6 = 15$$

Now let's compute g when S has 2 elements

$$\begin{aligned} g(2, \{3,4\}) &= \min\{C_{23} + g(3, \{4\}), C_{24} + g(4, \{3\})\} \\ &= \min(9 + 20, 10 + 15) = \min(29, 25) = 25 \end{aligned}$$

$$\begin{aligned} g(3, \{2,4\}) &= \min\{C_{32} + g(2, \{4\}), C_{34} + g(4, \{2\})\} \\ &= \min(13 + 18, 12 + 13) = \min(31, 25) = 25 \end{aligned}$$

$$\begin{aligned} g(4, \{2,3\}) &= \min\{C_{42} + g(2, \{3\}), C_{43} + g(3, \{2\})\} \\ &= \min(8 + 15, 9 + 18) = \min(23, 27) = 23 \end{aligned}$$

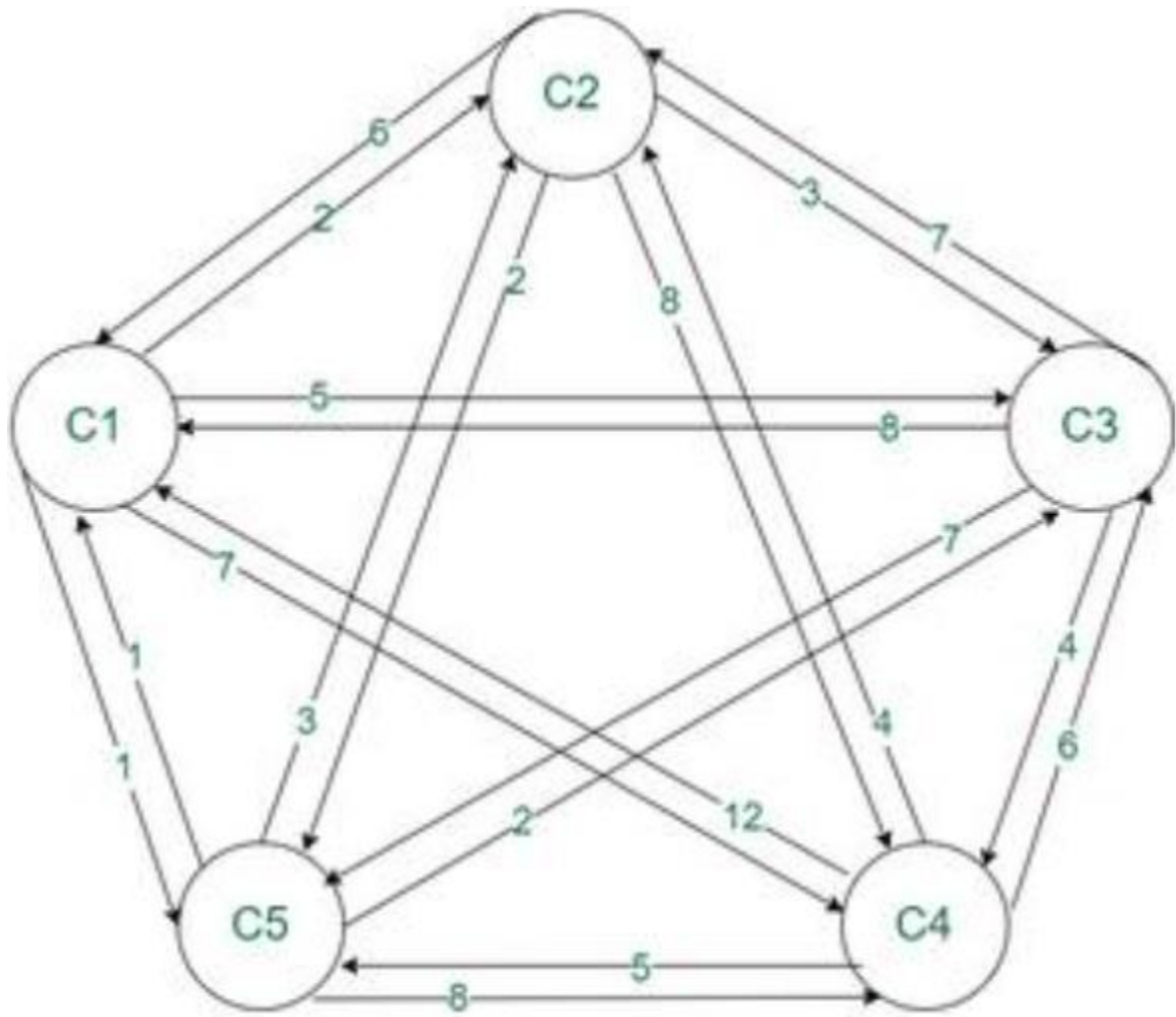
Now let's compute g when S has 3 elements and start vertex is 1

$$\begin{aligned} g(1, \{2,3,4\}) &= \min\{C_{12} + g(2, \{3,4\}), C_{13} + g(3, \{2,4\}), C_{14} + g(4, \{2,3\})\} \\ &= \min(10 + 25, 15 + 25, 20 + 23) \\ &= \min(35, 40, 43) = 35 \end{aligned}$$

Hence minimum travel cost is 35

The route is : 1,2,4,3,1

Exercise: Apply dynamic programming algorithm to solve the TSP problem for following graph assuming C1 as start vertex.



Longest Common Subsequence (LCS)

- A **subsequence** of a sequence, S is also a sequence where each element in the subsequence are from S and in increasing order of indices.
 - $S = \langle x_1, x_2, x_3, \dots, x_m \rangle$
 - $\langle x_1 \rangle, \langle x_1, x_2 \rangle, \langle x_1, x_2, x_4 \rangle, \langle x_1, x_4, x_m \rangle$ are some subsequences of S
 - $\langle x_2, x_1 \rangle, \langle x_4, x_2, x_3 \rangle$ are not subsequences of S
- **Common Subsequence** of two sequences is the sub subsequence common to both sequences
 - $S1 = \langle A, B, B, A, B, B \rangle$
 - $S2 = \langle B, A, A, A, B, B \rangle$
 - Some common subsequences of S1 and S2 are : $\langle A \rangle, \langle A, B \rangle, \langle A, A, B \rangle, \langle A, A, B, B \rangle, \langle A, B, B \rangle$
 - Longest common subsequence : $\langle A, A, B, B \rangle$
- The LCS length can be computed by using dynamic programming approach.
- Recurrence formula :

Let $C[i, j]$ be the length of an LCS of the sequences X_i and Y_j then

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

If sequence X is of length m and sequence Y is of length n then $c[m, n]$ gives length of LCS

Algorithm

```

LCS_Length(X,Y){
    m = X.length;
    n = Y.length;
    for(i=0;i<=m;i++)
        c[i,0] = 0;
    for(j=0;j<=n;j++)
        c[0,j] = 0;
    for(i = 1;i<=m;i++){
        for(j=1;j<=n;j++){
            if(X[i]==Y[j]){
                c[i,j] = c[i-1,j-1]+1;
                b[i,j] = "upleft";
            }
            else if (c[i-1, j]>= c[i, j-1]){
                c[i,j] = c[i-1,j];
                b[i,j] = "up";
            }
            else{
                c[i,j] = c[i,j-1];
                b[i,j] = "left";
            }
        }
    }
    return b and c;
}

```

Analysis

- Initialization of m and n happens in $O(1)$
- First loop takes $O(m)$ and second loop takes $O(n)$
- Third loop runs m times and the inner loop runs n times hence it takes $O(mn)$ times
- Therefore complexity of this algorithm is $O(mn)$

The optimal solution can be obtained from the table given by b[m,n] table. Firstly look at b[m,n] and trace through the table following the values(i.e up, upleft and left)

```

PrintLCS(b, X, i, j){
    if(i == 0 || j == 0) {
        return;
    }
    if(b[i,j]== "upleft") {
        PrintLCS(b, X, i-1,j-1);
        print X[i];
    }
    else if(b[i,j]== "up") {
        PrintLCS(b, X, i-1,j);
    }
    else{
        PrintLCS(b, X, i,j-1);
    }
}

```

Initial call PrintLCS(b, X, m,n);

Example:

Consider the character Sequences X=abbab, Y=aabaabb

Xi \ Yj			a	a	b	a	a	b	b
		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
a	1	0	1 upleft	1 upleft	1 left	1 upleft	1 upleft	1 left	1 left
b	2	0	1 up	1 up	2 upleft	2 left	2 left	2 upleft	2 upleft
b	3	0	1 up	1 up	2 upleft	2 up	2 up	3 upleft	3 upleft
a	4	0	1 upleft	2 upleft	2 up	3 upleft	3 upleft	3 up	3
b	5	0	1 up	1 up	3 upleft	3 up	3 up	4 upleft	4 upleft

Length of LCS = $C[m,n] = 4$

The LCS is : abbb

Minimum Edit Distance Problem

- Minimum edit distance is the minimum number of editing operations (insertion, deletion, substitution) required to convert one string into another.
- Dynamic programming approach can be used to find the minimum edit distance.

Recurrence formula:

Let $D[i,j]$ be the minimum edit distance of two strings X_i (source) and Y_j (target). The costs for insertion and deletion are considered as 1 and cost for substitution is considered as 2.

$$D[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ and } j = 0 \\ i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ \min \begin{cases} D[i-1,j] + 1 \\ D[i,j-1] + 1 \\ D[i-1,j-1] + \begin{cases} 2 & \text{if } i,j > 0 \text{ and } X[i] \neq Y[j] \\ 0 & \text{if } i,j > 0 \text{ and } X[i] = Y[j] \end{cases} \end{cases} & \text{if } i,j > 0 \end{cases}$$

If the source string X is of length m and the target string Y of length n then $D[m,n]$ gives the minimum edit distance.

Here,

$$\begin{aligned} D[i-1,j] + 1 & \text{ represents cost of deletion} \\ D[i,j-1] + 1 & \text{ represents cost of insertion} \\ D[i-1,j-1] + \begin{cases} 2 & \text{if } i,j > 0 \text{ and } X[i] \neq Y[j] \\ 0 & \text{if } i,j > 0 \text{ and } X[i] = Y[j] \end{cases} & \text{ represents cost of substitution} \end{aligned}$$

Algorithm

```

EditDistance(X,Y){
    m = X.length;
    n = Y.length;
    D[0,0] = 0;
    for(i=1;i<=m;i++)
        D[i,0] = i;
    for(j=1;j<=n;j++)
        c[0,j] = j;
    for(i = 1;i<=m;i++){
        for(j=1;j<=n;j++){
            D[i,j] = min( D[i-1,j]+1, D[i,j-1]+1, D[i-1,j-1]+(X[i]==X[j]?0:1));
        }
    }
    return D;
}

```

Note: The minimum edit distance is also known as Levenshtein distance

Analysis

$O(mn)$ [similar explanation as in LCS]

To compute the edit operations following pseudocode can be used

```

EditOperations(D,X,Y,i,j){
    if(i == 0 and j == 0) {
        return;
    }
    else if(i!=0 and D[i,j] == D[i-1,j]+1){
        print delete X[i];
        EditOperations(D,X,Y,i-1,j);
    }
    else if(j!=0 and D[i,j] == D[i,j-1]+1){
        print insert Y[j];
        EditOperations(D,X,Y,i,j-1);
    }
    else{
        if(x[i] == Y[j]){
            cost = 0;
        }else {
            cost = 2;
        }
        if(cost! = 0 and D[i,j] == min( D[i-1,j]+1, D[i,j-1]+1+ D[i-1,j-1]+cost)){
            print replace X[i] by Y[j];
            EditOperations(D,X,Y,i-1,j-1);
        }else{
            return;
        }
    }
}

Initial Call: EditOperations(D,X,Y,m,n);

```

Example:

X= stall , Y = table (i.e. calculating minimum edit distance to convert “stall” into “table”)

		t	a	b	l	e
X _i \ Y _j	0	1	2	3	4	5
0	0	←1	←2	←3	←4	←5
s	1	1↑	↖2↑	↖3↑	↖4↑	↖5↑
t	2	2↑	↖1	←2	←3	←4
a	3	3↑	2↑	↖1	↖2	←3
l	4	4↑	3↑	2↑	↖3↑	↖2
l	5	5↑	4↑	3↑	↖4↑	↖3↑

When $i = j = 0$, $D[0,0] = 0$

When $i = 0$, $D[0,j] = j$

When $j = 0$, $D[i,0] = i$

$D[1,1] = \min(D[0,1]+1, D[1,0]+1, D[0,0] + 2) = \min(1+1,1+1,0+2) = \min(2,2,2) = 2$

$D[1,2] = \min(D[0,2]+1, D[1,1]+1, D[0,0] + 2) = \min(2+1,2+1,1+2) = \min(3,3,3) = 3$

Similarly,

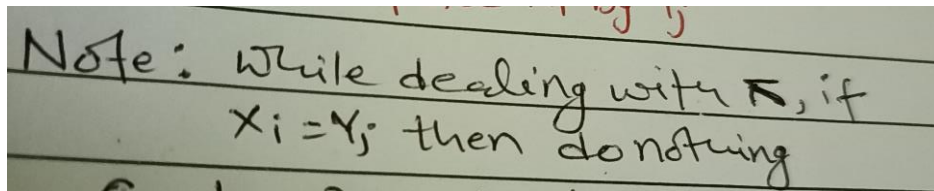
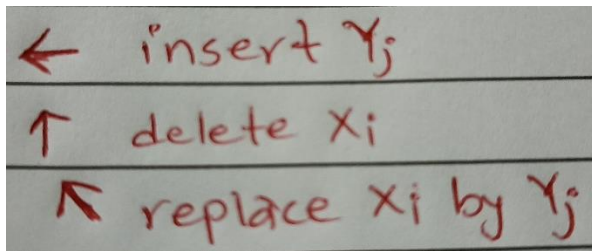
$D[1,3] = 4, D[1,4] = 5, D[1,5] = 6$

$D[2,1] = \min(D[1,1]+1, D[2,0]+1, D[1,0] + 0) = \min(2+1,2+1,1+0) = \min(3,3,1) = 1$

Similarly other values are filled in the table.

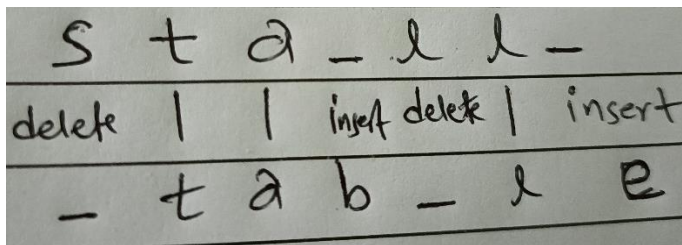
,

The arrows have following meanings:



The minimum edit distance is $D[m,n] = D[5,5] = 4$

Following the arrows (red and bold) starting from $D[m,n] = D[5,5]$ we can find the operation details as follows. [NOTE: we can also follow other possible paths as directed by the arrows]



- delete s // cost = 1
- do nothing cost = 0
- do nothing // cost = 0
- insert b // cost = 1
- delete l // cost = 1
- do nothing // cost = 0
- e is inserted // cost = 1
- Total cost = 4

NOTE: In this example all possible arrows are shown in table

Concept of memoization

- Memoization is a term describing an optimization technique where you cache previously computed results, and return the cached result when the same computation is needed again.
- Memoization ensures that a method doesn't run for the same inputs more than once by keeping a record of the results for the given inputs
- Memoization is top down approach
- Dynamic programming can also be implemented using memoization. With memoization, we implement the algorithm recursively, but we keep track of all of the subproblems. If a subproblem which is already solved reappears then the solution is obtained from the record. If an unsolved subproblem appears then it is solved and added to the record having solution of list of subproblems. Each subsequent time that the subproblem is encountered, the value stored in table is simply looked up and returned.
- Memoization offers the efficiency of dynamic programming. It maintains the top down recursive strategy.
- A memoized recursive algorithm maintains an entry in a table for the solution to each subproblem. Each table entry initially contains a special value to indicate that the entry has yet to be filled in. When the subproblem is first encountered as the recursive algorithm unfolds, its solution is computed and then stored in the table. Each subsequent time that we encounter this subproblem, we simply look up the value stored in the table and return it.

Dynamic programming vs Memoization

- Both Memoization and Dynamic Programming solves individual subproblem only once.
- Memoization uses recursion and works top-down, whereas Dynamic programming moves in opposite direction solving the problem bottom-up.
- When you solve a dynamic programming problem using tabulation you solve the problem "bottom up", i.e., by solving all related sub-problems first, typically by filling up an n-dimensional table. Based on the results in the table, the solution to the "top" / original problem is then computed.
- If you use memoization to solve the problem you do it by maintaining a map of already solved sub problems. You do it "top down" in the sense that you solve the "top" problem first (which typically recurses down to solve the sub-problems).

- In general practice, if all subproblems must be solved at least once, a bottom-up dynamic-programming algorithm usually outperforms the corresponding top-down memoized algorithm by a constant factor, because the bottom-up algorithm has no overhead for recursion and less overhead for maintaining the table. Moreover, for some problems we can exploit the regular pattern of table accesses in the dynamic programming algorithm to reduce time or space requirements even further. Alternatively, if some subproblems in the subproblem space need not be solved at all, the memoized solution has the advantage of solving only those subproblems that are definitely required.

Assignment:

Explain how n th term of Fibonacci sequence is computed using memoization technique? And what about following bottom up dynamic programming technique for the same? Discuss with illustrative example.